

PROPOSTA TÉCNICA

Smart Tennis Arena

Projeto Smash

Projeto Interdisciplinar: IoT e Robótica
Ciência da Computação · 6º Semestre · FECAP

EQUIPE

Felipe Vallim Soares (24026060)
Gustavo Oliveira Demetrio (24026213)
Guilhermy Mariano Lisboa Garcia (23025371)
Saulo Pereira de Jesus (24026095)

1. Descrição Técnica do Problema e Delimitação do Escopo

O problema

O treinamento de precisão contra o “paredão” é a forma mais acessível de praticar tênis, porém carece totalmente de geração de dados. O jogador não tem registro de sua taxa de acertos, consistência da força aplicada ou tempo de reação. As soluções comerciais existentes (como máquinas lançadoras ou sistemas de visão computacional em quadras) apresentam custos proibitivos e infraestrutura complexa, inacessíveis ao jogador amador e a pequenos clubes.

Escopo da solução (Smash)

O projeto consiste no desenvolvimento de um sistema IoT de baixo custo que instrumenta uma parede comum, transformando impactos físicos em telemetria. O hardware baseia-se em uma matriz física 3x3 (9 alvos em placas de MDF), onde cada quadrante é monitorado por um sensor piezoelétrico.

O sistema restringe-se a detectar qual dos 9 alvos foi atingido e a intensidade relativa da vibração gerada pela colisão. O fluxo exclui triangulação milimétrica complexa e adota uma abordagem de comparação de picos de tensão via microcontrolador ESP32. Os dados de impacto são enviados no formato JSON para um aplicativo mobile (React Native), responsável pela interface gamificada e persistência em um backend (Node.js/NestJS).

2. Diagrama de Blocos do Sistema

O sistema é modular e dividido nos seguintes blocos funcionais:

- **Bloco Físico (Entrada):** matriz 3x3 de alvos independentes em MDF, isolados por espaçadores de espuma para controle de ressonância.
- **Bloco de Sensoriamento e Condicionamento:** 9 pastilhas piezoelétricas (27 mm) fixadas atrás dos alvos. Cada pastilha conta com um circuito de proteção (1 resistor 1 MΩ para escoamento de carga e 2 diodos 1N4148 para grampeamento de tensão). O cabeamento é feito via cabos de rede RJ45 (Cat5) para cancelamento de ruído.
- **Bloco de Multiplexação:** circuito integrado 4051 (multiplexador de 8 canais). Os 8 primeiros alvos são roteados pelo mux para o ADC1 do ESP32, e o 9º alvo é ligado diretamente a um pino analógico livre do ADC1.
- **Bloco de Atuadores:** 1 módulo de LED (ou segmento de fita) instalado em cada alvo, fornecendo feedback visual instantâneo ao jogador (requisito RF08).
- **Bloco de Processamento IoT:** microcontrolador ESP32 (38 pinos) executando firmware em C++ (framework Arduino) para varredura e processamento do sinal digitalizado.
- **Bloco de Comunicação:** módulo sem fio integrado ao ESP32. Operará via Bluetooth Serial no MVP e evoluirá para Wi-Fi via protocolo MQTT (broker Mosquitto).
- **Bloco de Aplicação e Persistência:** aplicativo mobile React Native (TypeScript) para a gamificação na ponta do usuário, integrado via API REST a um servidor Node.js/NestJS com banco de dados PostgreSQL.

3. Definição do Fluxo de Dados

O fluxo (Entrada → Processamento → Saída → Registro) ocorre em tempo real, da seguinte maneira:

- **Entrada:** a bola colide com a placa de MDF. A vibração deforma o cristal piezoelétrico, gerando um pico de tensão condicionado pelo circuito de proteção.

- **Processamento:** o ESP32 realiza uma varredura contínua dos canais via CI multiplexador. Ao identificar uma tensão superior ao limiar configurado (threshold), o ESP32 abre uma janela de leitura de 20 ms medindo todos os canais. O canal que registrar a maior amplitude é declarado o vencedor, eliminando falsos positivos decorrentes de vazamento mecânico.
- **Saída:** o firmware empacota o evento e o transmite via Serial/Bluetooth (ou MQTT).
- **Registro:** o aplicativo mobile recebe o payload, converte o t_ms para o instante real da sessão em andamento, atualiza a interface gamificada e efetua o POST na API do backend para persistência no PostgreSQL.

Contrato de dados (JSON)

```
{"alvo": 5, "linha": 2, "coluna": 2, "intensidade": 1840, "t_ms": 42315, "sessao": "a3f1"}
```

4. Requisitos Funcionais e Não Funcionais

Requisitos Funcionais (RF)

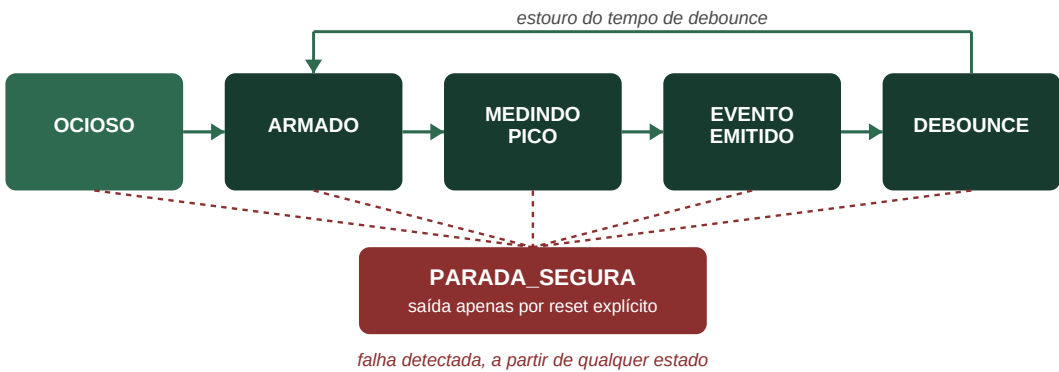
ID	Descrição
RF01	Aquisição: o sistema embarcado deve realizar a leitura sequencial de 9 sensores analógicos através de multiplexação e enviar a medição (evento de impacto).
RF02	Comunicação: o sistema deve transmitir dados ao aplicativo via conexão sem fio (Bluetooth ou Wi-Fi/MQTT).
RF03	Gamificação: o aplicativo mobile deve iniciar as sessões, calcular pontuação em tempo real baseada nos alvos atingidos e exibir mapas de calor.
RF04	Parada Segura: implementação de gatilho em software (modo falha) que cessa o envio/recebimento de comandos até que o sistema seja explicitamente reiniciado, conforme exigência RF22.

Requisitos Não Funcionais (RNF)

ID	Descrição
RNF01	Desempenho da Varredura: o processador ESP32 a 240 MHz deve varrer os 9 canais em menos de 1 milissegundo, garantindo o registro de impactos de alta velocidade.
RNF02	Tolerância a Falhas Físicas: a inteligência de software deve mitigar o vazamento de vibração entre os alvos por meio da lógica de “canal de maior pico”, evitando duplicação de dados.
RNF03	Padronização e Tipagem: a comunicação entre hardware, mobile e backend deverá obedecer a interfaces estritas (TypeScript), prevenindo quebras por eventos malformados.

5. Modelagem do Comportamento (Máquina de Estados)

Para a análise de Linguagens Formais, o comportamento do firmware será regido por uma máquina de estados finitos que rege o ciclo de leitura de cada impacto:



- **Estado 1 – OCIOSO:** ESP32 ligado, aguardando conexão da sessão.
- **Estado 2 – ARMADO:** ESP32 varre continuamente os 9 canais analógicos. Transição: cruza o limiar de tensão (threshold) → vai para MEDINDO_PICO.
- **Estado 3 – MEDINDO_PICO:** grava as intensidades máximas de todos os canais por uma janela de 20 ms. Transição: estouro de tempo (20 ms) → seleciona o vencedor e vai para EVENTO_EMITIDO.
- **Estado 4 – EVENTO_EMITIDO:** monta a estrutura JSON e dispara a transmissão (Bluetooth/MQTT). Transição: imediata após envio → vai para DEBOUNCE.
- **Estado 5 – DEBOUNCE:** ignora leituras por um período pré-determinado (ex.: 200 ms) para não registrar as reverberações da mesma batida. Transição: estouro do tempo de espera → retorna para ARMADO.
- **Estado 6 – PARADA_SEGURA:** alcançável a partir de qualquer estado por acionamento de emergência (falha detectada). Interrompe todas as medições e comandos para os atuadores. Transição: saída apenas por reinício explícito (reset) do sistema.

6. Product Backlog Inicial (Framework Scrum)

O escopo foi dividido em épicos técnicos e priorizado para validações sucessivas sem pular etapas:

Épico 1 — Validação de Hardware e Condicionamento

- **US01** – Como engenheiro, desejo ligar uma pastilha piezoelétrica na protoboard com o circuito de grampeamento (1 MΩ + 1N4148) para calibrar a leitura da tensão gerada por batidas.
- **US02** – Como engenheiro, desejo colar o sensor no MDF e ler o sinal analógico no ESP32 para verificar a queda de tensão após a absorção da madeira.
- **US03** – Como desenvolvedor de hardware, desejo montar 2 alvos fixados lado a lado para medir e isolar o vazamento de vibração mecânica entre eles.

Épico 2 — Firmware e Multiplexação

- **US04** – Como programador (C++), desejo integrar o DIP-16 ao ESP32 para conseguir ler alternadamente os 9 alvos usando o conversor ADC1.
- **US05** – Como desenvolvedor, desejo programar a máquina de estados (limiar → medição 20 ms → debounce) para imprimir o JSON com o canal vencedor no monitor serial.

Épico 3 — Ecossistema de Software e Integração

- **US06** – Como desenvolvedor mobile, desejo que o React Native ouça a porta Bluetooth (ou tópico MQTT) e console os eventos enviados pelo ESP32.
- **US07** – Como jogador, desejo visualizar na tela do aplicativo um indicativo visual instantâneo confirmando o alvo que acabei de acertar.
- **US08** – Como desenvolvedor backend, desejo estruturar os schemas do PostgreSQL e as rotas do NestJS para persistir o histórico (payload) após a conclusão do treino.